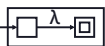
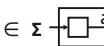
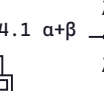


Alfabeto (eq. Σ): Conjunto finito no vacío de símbolos Cadena[Σ]: Secuencia finita de símbolos de Σ Σ^*: Conjunto de todas las cadenas sobre Σ Lenguaje[Σ]: Subconjunto de Σ^* Lenguaje Regular: $\forall$ lenguaje sssi $\exists M(\text{AFRD}) \mid L=L(M)$ Sean L_1 y L_2 dos lenguajes sobre un alfabeto Σ: - Unión: $L_1 \cup L_2 = \{w \mid w \in L_1 \vee w \in L_2\}$ - Intersección: $L_1 \cap L_2 = \{w \mid w \in L_1 \wedge w \in L_2\}$ - Complemento: $L_1' = \{w \mid w \in \Sigma^* \wedge w \notin L_1\}$ - Concatenación: $L_1 \circ L_2 = \{w_1w_2 \mid w_1 \in L_1 \wedge w_2 \in L_2\}$ - Estrella de Kleene: $L_1^* = \{w_1w_2...w_n \mid w_i \in L_1, n \geq 0\}$	Autómata Finito (AF): Modelo que captura aspectos de una computadora - AF Reconocedor (AFR): Reconoce Lenguajes Regulares - AFR Determinista (AFRD): Transiciones $\forall a \in \Sigma$ - AFR No Determinista (AFRND): Transiciones $(*) \forall a \in \Sigma$ - AFRND Con Transiciones λ (AFRND-λ): Trans. $(*) \forall a \in \Sigma$ y λ - Autómata Finito Traductor (AFT): Poseen input y output - AFT De Moore: Salidas asoc. a estados - AFT De Mealy: Salidas asoc. a transiciones
AFRD: Quintupla $M = (S, \Sigma, \delta, s_0, F)$, donde: - S: Conjunto Finito de Estados ($S \neq \emptyset$) - Σ: Alfabeto del Lenguaje Regular - δ: Función de Transición ($S \times \Sigma \rightarrow S$) - s_0: Estado Inicial ($s_0 \in S$) - F: Conjunto de Estados Finales ($F \subseteq S$) Generalización de δ^*: $S \times \Sigma^* \rightarrow S$ ($a \in \Sigma \wedge w \in \Sigma^*$) - $\delta^*(q, \lambda) = q$, $\delta^*(q, aw) = \delta^*(\delta(q, a), w)$ - eq: $\delta^*(q, \lambda) = q$, $\delta^*(q, wa) = \delta(\delta^*(q, w), a)$ Reconoce el Lenguaje: $L(M) = \{w \mid \delta^*(s_0, w) \in F\}$ AFRND redefine $\delta \mid \delta$:Función de Transición ($S \times \Sigma \rightarrow 2^S$) Generalización de δ^*: $S \times \Sigma^* \rightarrow 2^S$: - $\delta^*(q, \lambda) = \{q\}$, $\delta^*(q, wa) = \{r_1, r_2, ..., r_m\}$, con: - $\delta^*(q, w) = \{p_1, p_2, ..., p_k\}$ y $U(k, i=1, \delta(p_i, a))=\{r_1, r_2, ..., r_m\}$ Reconoce el Lenguaje: $L(M) = \{w \mid \delta^*(s_0, w) \cap F \neq \emptyset\}$ AFRND-λ redefine $\delta \mid \delta$:Función de Transición ($S \times (\Sigma \cup \{\lambda\}) \rightarrow 2^S$)	AFT De Moore: Séxtupla $M = (S, \Sigma, \Gamma, \delta, s_0, f_0)$, donde: - S: Conjunto Finito de Estados ($S \neq \emptyset$) - Σ, Γ: Alfabetos de Entrada y Salida - δ: Función de Transición ($S \times \Sigma \rightarrow S$) - s_0: Estado Inicial ($s_0 \in S$) - f_0: Función de Salida ($S \rightarrow \Gamma^*$) AFT De Mealy: (Idem Moore) donde: - f_0: Función de Salida ($S \times \Sigma \rightarrow \Gamma^*$) Expresiones Regulares (ER) Conjunto de cadenas definidas sobre un alfabeto (Σ) $A = \Sigma \cup \{ (,), *, +, \lambda, \emptyset \}$, def. recursivamente como sigue (Sean α y β ERs): - $\lambda, \emptyset, \forall a \in \Sigma$ son ERs - $(\alpha), \alpha \cdot \beta, \alpha + \beta$ y α^* tambien son ERs Reconoce el Lenguaje: - $L(\lambda) = \{\lambda\}$, $L(\emptyset) = \emptyset, \forall a \in \Sigma, L(a) = \{a\}$ - $L(\alpha \cdot \beta) = L(\alpha) \cdot L(\beta), L(\alpha + \beta) = L(\alpha) \cup L(\beta), L(\alpha^*) = L(\alpha)^*$ Que las fuerzas del cielo nos acompañen
Teorema De Kleene: Un lenguaje (L) es expresable con una ER sssi es reconocido por un AFR Parte 1: ER $\rightarrow$ AFR 1. λ  2. $\emptyset$  3. $a \in \Sigma$  4.1 $\alpha + \beta$  4.2 $\alpha \cdot \beta$  4.3 α^*  Parte 2: AFR $\rightarrow$ ER: $w s = \beta + w s \vee \beta \vee \gamma^*, \alpha + \beta = \beta + \alpha, \alpha(\beta + \gamma) = \alpha\beta + \alpha\gamma, (\beta + \gamma)\alpha = \beta\alpha + \gamma\alpha, \lambda\alpha = \alpha$ Dado un AFR (M) existe una Gramática Regular (G) $\mid L(M)=L(G)$, y viceversa Parte 1: G $\rightarrow$ AFR 1. $S = V_n \cup \{X\}$ 2. $\Sigma = V_t$ 3. YKH 4. $s_0 = S$ 5. $F = \{X, (S \text{ si } \exists S \rightarrow \lambda)\}$ Parte 2: AFR $\rightarrow$ G 1. $V_n = S$ 2. $V_t = \Sigma$ 3. $S = s_0$ 4. YKH Un algoritmo es correcto si para cualquier input legal finaliza y produce el output deseado. Las pruebas de correctitud no pueden ser automatizadas. Para probar correctitud parcial son asociadas aserciones (sentencias sobre el estado de la ejecución) a puntos específicos del algoritmo Precondiciones: Aserciones que deben ser válidas antes de ejecutarse un algoritmo o subrutina Poscondiciones: Aserciones que deben ser válidas luego de ejecutarse un algoritmo o subrutina Tripla de Hoare: $\{Q\}P\{R\}$ Q: precondición para el programa P R: poscondición para el programa P Condicionales: $\{Q \wedge B\}P_1\{R\} \vee \{Q \wedge \neg B\}P_2\{R\} \rightarrow \{Q\}si\{R\}$ Bucles: $\{Q \wedge B\}P\{Q\} \rightarrow \{Q\}si\{Q \wedge \neg B\}$	Gramática ($\emptyset$): Cuadrupla $G = (V_n, V_t, S, P)$, donde: - V_n: Conjunto de Símbolos No Terminales - V_t: Conjunto de Símbolos Terminales - S: Símbolo Inicial de la Gramática - P: Conjunto Finito de Producciones ($w_1 \rightarrow w_2 \mid w_1, w_2 \in (V_n \cup V_t)^* \wedge \exists a \mid a \in V_n$) Reconoce el Lenguaje: $L(G) = \{w \in V_t^* \mid S \rightarrow^+ w\}$ Clases: - Estructura de Frase ($\emptyset$): Sin restricciones - Sensible al Contexto (1): $\forall \alpha \rightarrow \beta \mid \alpha \leq \beta \vee S \rightarrow \lambda$ - Libre de Contexto (2): (Idem 1) con $\alpha \in V_n$ - Regular (3): (Idem 2) $\beta = t/tW \mid t \in V_t$ y $W \in V_n$ Minifiación AF: Algoritmo que dado un AF (M), genera un AF (M'), donde: $L(M) = L(M')$ y $M'[S] < M[S]$ (M minificado) o $M'[S] = M[S]$ (M es mínimo). El algoritmo es aplicable a AFTs De Moore y a AFRD (donde $\emptyset$-eq tendra F y a $S-F$): 1- $S - \{\text{estados inalcanzables en } S\} \rightarrow T$ 2- Particionar T en clases de estados $\emptyset$-eq 3- $k:=\emptyset$ 4- Repetir - Determinar clases $(k+1)$-eq refinando las clases k-eq si, s_j son $(k+1)$-eq sssi si, s_j son k-eq $\wedge \delta(s_i, a), \delta(s_j, a)$ son k-eq $k++$ Hasta Clases$((k+1)\text{-eq})=\text{Clases}(k\text{-eq})$
Demostraciones: - Complemento: Sea L regular ppq L' es regular: Por definicion, $\exists M = (S, \Sigma, \delta, s_0, F) \mid L(M) = L$. Propongo $M' = (S, \Sigma, \delta, s_0, F')$ $\mid F' = S-F$. Ppq $L(M') = L'$: Sea $w \in L' \rightarrow w \notin L \rightarrow w \notin L(M) \rightarrow \delta^*(s_0, w) \notin F \rightarrow \delta^*(s_0, w) \in F' \rightarrow w \in L(M') \rightarrow$ Por definicion de leguaje regular concluyo asi L' regular. - Unión: Sea L_1 y L_2 regular ppq $L_1 \cup L_2$ es regular: Por definicion de lenguaje regular, $\exists M_1 \vee M_2 \mid L(M_1) = L_1$ y $L(M_2) = L_2$. Por Teorema de Kleene (Parte 1), $\exists E_1$ y $E_2 \mid L(E_1) = L(M_1)$ y $L(E_2) = L(M_2)$. Por definicion de ER, $\exists E_3 \mid E_3 = E_1 + E_2$. Por Teorema de Kleene (Parte 2), $\exists M_3 \mid L(M_3) = L(E_3)$. Por definicion de leguaje regular, $\exists L_3 \mid L_3 = L(M_3)$. Concluyo asi $L_1 \cup L_2 = L_3$ regular. - Intersección: Sea L_1 y L_2 regular ppq $L_1 \cap L_2$ es regular: Siendo los lenguajes regulares cerrads bajo Union y Complemento, y por Ley de De Morgan para conjuntos $((A \cup B)' = A' \cap B') \rightarrow ((L_1' \cup L_2')' = L_1 \cap L_2 \rightarrow$ Concluyo asi que $L_1 \cap L_2$ es regular.Demostraciones - Concatenación:Sea L_1 y L_2 regular ppq $L_1 \cdot L_2$ es regular: Por definicion de lenguaje regular, $\exists M_1 \vee M_2 \mid L(M_1)=L_1$ y $L(M_2)=L_2$. Por Teorema de Kleene (Parte 1), $\exists E_1$ y $E_2 \mid L(E_1)=L(M_1)$ y $L(E_2)=L(M_2)$. Por definicion de ER, $\exists E_3 \mid E_3=E_1 \cdot E_2$. Por Teorema de Kleene (Parte 2), $\exists M_3 \mid L(M_3)=L(E_3)$. Por definicion de leguaje regular, $\exists L_3 \mid L_3=L(M_3)$. Concluyo asi $L_1 \cdot L_2=L_3$ regular. - Estrella de Kleene:Sea L_1 regular ppq L_1^* es regular: Por definicion de lenguaje regular, $\exists M_1 \mid L(M_1)=L_1$. Por Teorema de Kleene (Parte 1), $\exists E_1 \mid L(E_1)=L(M_1)$. Por definicion de ER, $\exists E_2 \mid E_2=E_1^*$. Por Teorema de Kleene (Parte 2), $\exists M_2 \mid L(M_2)=L(E_2)$. Por definicion de leguaje regular, $\exists L_2 \mid L_2=L(M_2)$. Concluyo asi $L_1^*=L_2$ regular.	Factos: (Verdades riales de la humanidad, factoriales, etc) - Todo lenguaje finito es regular - Sea L_r un lenguaje regular y $L \mid L \subseteq L_r, L$ no tiene por que ser un lenguaje regular - Todo estado de un AF es alcanzable si es el inicial o alcanzable desde un alcanzable - Todo estado inalcanzable es prescindible - Todo AFT De Mealy es equivalente a un AFT De Moore, salvo en traducciones para λ que no sean λ - YAKH (You Already Know How): Demasiado largo, no me alcanza el margen
Lenguajes NO Regulares (EJ): $L_1=\{a^i \mid i=3^n, \text{ con } n \geq 0\}$ $L_2=\{a^n b^n \mid \text{ con } n \geq 0\}$ $L_3=\{a^n b^m c^p \mid m=n+p, \text{ con } n, m, p \geq 0\}$ $L_4=\{a^i b^j c^k \mid 2i=3j+k, \text{ con } i, j, k \geq 0\}$ $L_5=\{(a+b)^n (b+c)^m d^{n+m} \mid n, m \geq 0\}$	